



Khulna University of Engineering and Technology

Project Report for MTE 3104

[Sessional on Microprocessor, Microcontroller and Interfacing]

Title - Blind Spot: Adaptive Staircase Lighting and Fall Prevention System

By

Group 4

[2231019]	Samia Akter Bithi
[2231020]	Estimam Faiyas Muhammad Nafis
[2231021]	Ariyan Alam
[2231023]	Julker Rahhim Samin
[2231024]	Md. Ratbin Khan Rohan

INDEX

Serial No.	Title	Page No
1	Objectives	3
2	Introduction	3
3	Required Components	4
4	Methodology	4
5	Result	11
6	Discussion	17
7	Expenditure	19
8	Application	19
9	Conclusion	20
10	Reference	21
11	Appendix	21

1. Objectives:

The objectives of this experiment is-

1. To implement predictive staircase illumination based on direction of travel.
2. To dynamically adjust staircase brightness using ambient light sensing.
3. To develop a two-stage fall detection and warning mechanism.
4. To implement remote caregiver notification using Wi-Fi and Blynk platform integration.
5. To implement a password protected door entry.
6. To gain practical experience in embedded systems, IoT communication, PWM control, and real time monitoring.

2. Introduction:

Modern residential buildings increasingly demand intelligent systems that can improve both security and safety while reducing energy consumption[10]. Conventional door locking systems and staircase lighting mechanisms often lack automation, adaptive control, and emergency response capabilities.

This project proposes an integrated smart access and staircase safety system based on the ATmega32A microcontroller[1]. The first section of the project focuses on access control, where users enter a predefined password using push-button switches. An OLED display provides visual feedback regarding password entry and system status[2], while a servo motor acts as the door locking mechanism.

After successful authentication, the staircase monitoring subsystem becomes active. PIR sensors installed at entry locations determine movement direction through sequential triggering[3]. IR break-beam sensors validate entry and exit events to improve detection reliability[4]. Based on movement direction, the system activates LED lighting zones ahead of the user, providing predictive illumination.

To enhance energy efficiency, an LDR continuously measures ambient light intensity[5], allowing the microcontroller to adjust LED brightness dynamically using PWM control[8]. Additionally, PIR sensors placed at intermediate stair locations monitor movement patterns for fall detection[3]. If motion ceases for a predetermined time interval, a buzzer warning is initiated. If the user remains motionless, the ESP8266 WiFi module transmits an emergency notification to caregivers through the Blynk platform[6][7].

The proposed system integrates security, intelligent lighting, energy management, and emergency assistance into a single low-cost embedded platform suitable for smart home applications.

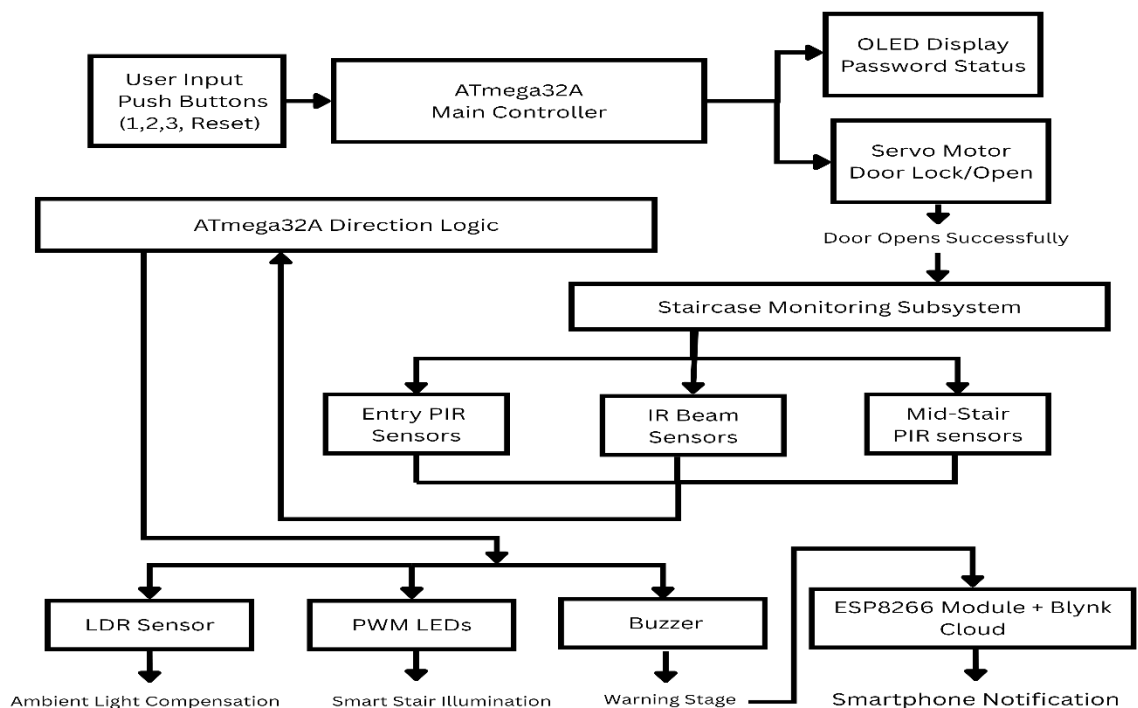
3. Required Components:

Table 1.1: Required components for the experiment.

SL no	Name of the components	Rating	Quantity
1	Lithium ion 3s	3.7 Volts dc,100 amp, 320 watt-hour	3
2	ATmega 32	(3.8-5.5)V, 10-15 MHz	1
3	AVR Programmer	(0-5)V, 500 mA	1
4	HC-SR501 PIR Sensors	(4.5-20)V dc, <50 μ A	2
5	IR Sensor	(3.3-5)V dc, (15-20) mA	2
6	LDR + Resistor	(3.3-5)V dc, (15-20) mA	1
7	LED	(1.8-5)V, (10-20) mA	4
8	OLED	(3.3-5)V dc, (20-40) mA	1
9	ESP32 Module	(3.3-5)V dc, 500 mA, 240 MHz	1
10	Buzzer	(5-12)V , <30mA	1
11	LM2596 Buck Converter	(0-30)V, (0-3) Amp	1
12	Servo motor	(5-12)V, (1-2) Amp	1
13	Push Buttons	-----	4
14	Bread Board	-----	2
15	Connecting wires	-----	As required

4. Methodology:

4.1 Block Diagram:



4.2 Algorithm:

START

Initialize ATmega32A peripherals

Initialize OLED Display

Initialize Servo Motor

Initialize PIR Sensors

Initialize IR Sensors

Initialize LDR

Initialize LEDs

Initialize Buzzer

Initialize ESP8266

Initialize Blynk connection

Door_State $\leftarrow$ LOCKED

WHILE (1)

 Display "Enter Password"

 Read button inputs

 IF Reset_Button is pressed THEN

 Execute Password_Reset()

 ENDIF

 IF Three digits are entered THEN

 IF Entered_Password == Stored_Password THEN

 Display "Access Granted"

 Rotate Servo to Unlock Position

 Door_State $\leftarrow$ OPEN

 Enable Staircase Monitoring

 ELSE

 Display "Access Denied"

Activate Error Indication
Clear Input Buffer

ENDIF

ENDIF

IF Door_State == OPEN THEN

Read Entry PIR Sensors
Read IR Beam Sensors

IF Sensor Sequence indicates UP movement THEN

Direction ← UP

Turn ON LED zones ahead

ELSE IF Sensor Sequence indicates DOWN movement THEN

Direction ← DOWN

Turn ON LED zones ahead

ENDIF

Read LDR Value

IF Ambient Light is LOW THEN

Increase PWM Duty Cycle

ELSE

Decrease PWM Duty Cycle

ENDIF

Read Mid-Stair PIR Sensor

IF No Motion Detected for Preset_Time THEN

 Activate Buzzer

 Start Warning_Timer

 IF No Motion During Warning_Timer THEN

 Send Emergency Alert

 through ESP8266 using Blynk

 ENDIF

ENDIF

IF IR Sensors Detect Exit Event THEN

 Turn OFF LEDs

 Reset Staircase Variables

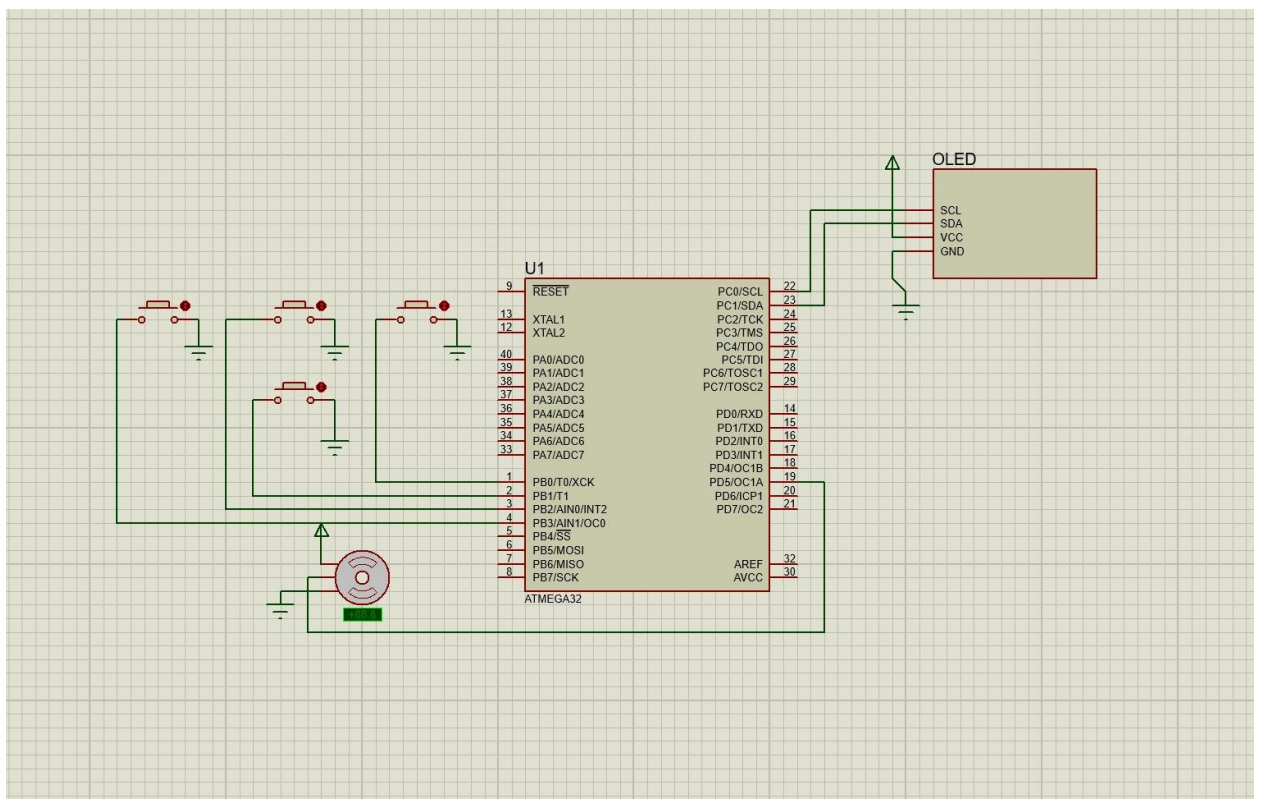
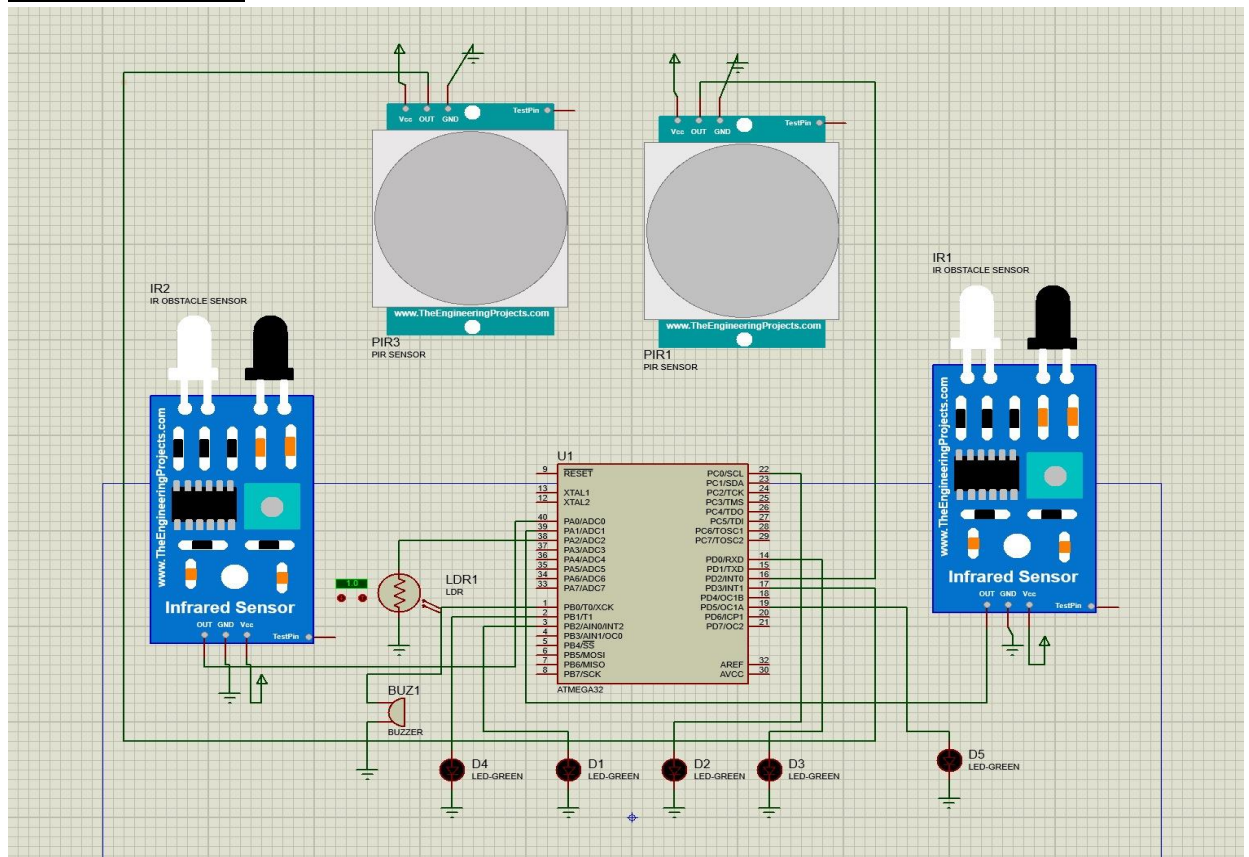
ENDIF

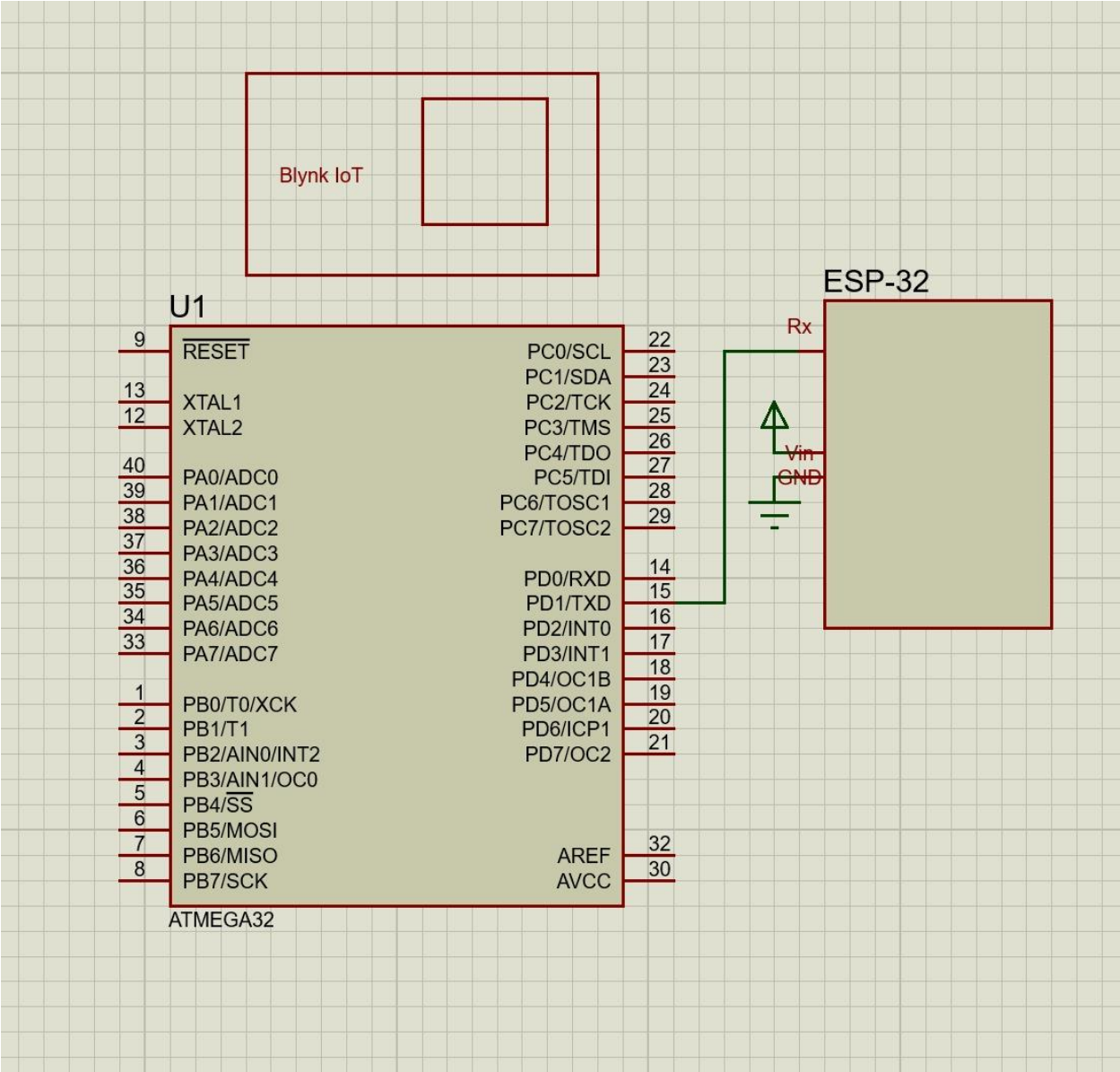
ENDIF

END WHILE

END

4.3 Circuit Diagram:





4.4 Description:

The proposed system operates through two major subsystems: door access control and intelligent staircase monitoring. The algorithm begins by initializing all peripherals connected to the ATmega32A microcontroller, including the OLED display, push buttons, servo motor, PIR sensors, IR break-beam sensors, LDR sensor, LED lighting zones, buzzer, and ESP8266 Wi-Fi module. The servo motor is initially positioned to keep the door in a locked state[1][6].

After initialization, the system continuously waits for user input. The OLED display prompts the user to enter a password using the three designated push buttons. A fourth push button is provided to reset the password when required. As the user enters the password, the microcontroller stores the input sequence temporarily and displays the entered digits or symbols on the OLED screen[2]. Once the password entry is completed, the ATmega32A compares the entered password with the password stored in memory.

If the entered password matches the stored password, the system grants access by displaying a confirmation message on the OLED display and rotating the servo motor to the predefined unlock position, thereby opening the door. Simultaneously, the staircase monitoring subsystem is activated. If the password is incorrect, an error message is displayed, access is denied, and the input buffer is cleared to allow another attempt.

Upon activation of the staircase subsystem, PIR sensors installed near the staircase entrance continuously monitor user movement[3]. The direction of movement is determined based on the sequential triggering pattern of these sensors. To improve reliability and reduce false detections, IR break-beam sensors are used to verify complete entry and exit events[4]. Once the direction of travel is identified, the microcontroller illuminates LED zones located ahead of the user's current position, providing predictive lighting and improving user convenience.

The ambient light intensity is measured continuously using the LDR sensor[5]. The analog output from the LDR is converted into a digital value using the internal Analog-to-Digital Converter (ADC) of the ATmega32A. Depending on the measured brightness level, the microcontroller adjusts the PWM duty cycle supplied to the staircase LEDs[8]. During daylight conditions, the LEDs operate at reduced brightness to conserve energy, while at night or under low-light conditions, higher duty cycles are applied to ensure adequate illumination.

For safety monitoring, PIR sensors positioned at intermediate locations on the staircase continuously observe user movement. If no movement is detected for a predetermined period, the system interprets the situation as a possible fall or emergency condition. In the first stage of the alert mechanism, a local buzzer is activated to warn the user and provide an opportunity to resume movement. If motion is still not detected after an additional waiting interval, the second stage is initiated. The ESP8266 Wi-Fi module communicates with the Blynk IoT platform and sends an emergency notification to a caregiver's smartphone, allowing timely assistance[6][7].

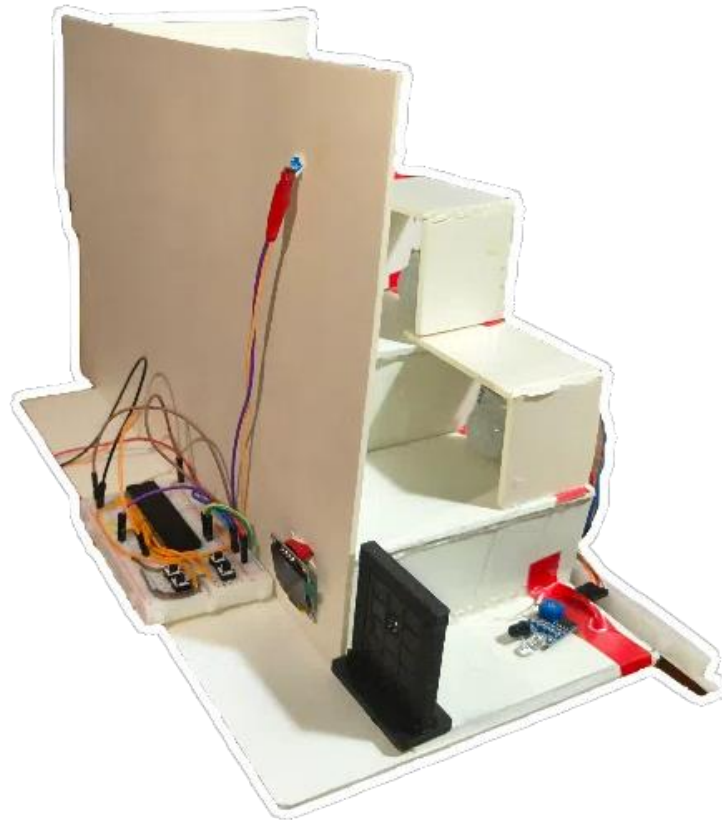
Finally, when the IR sensors detect that the user has completely exited the staircase, all lighting zones are turned off, monitoring variables are reset, and the system returns to its standby state,

awaiting the next authenticated user. This continuous monitoring and adaptive control approach ensures enhanced security, energy efficiency, and safety for residential staircase applications.

5. Result:

5.1 Photo of System:





5.2 Quantitative Analysis:

5.2.1 UART Baud Rate Register Calculation:

$$UBRR = (F_{cpu} / (16 \times \text{Baud})) - 1$$

$$\begin{aligned} UBRR &= \frac{1000000}{16 \times 4800} - 1 \\ UBRR &= 13.02 - 1 \\ UBRR &\approx 12 \end{aligned}$$

Actual baud rate:

$$\begin{aligned} \text{Baud}_{actual} &= \frac{1000000}{16(12 + 1)} \\ \text{Baud}_{actual} &= 4807.69 \text{ bps} \end{aligned}$$

Percentage Error:

$$\begin{aligned} \text{Error} &= \frac{4807.69 - 4800}{4800} \times 100 \\ \text{Error} &= 0.16\% \end{aligned}$$

UART communication error is only 0.16%, which is well within acceptable limits.

5.2.2 PWM Frequency Calculation:

Prescaler = 8

Fast PWM 8-bit:

$$f_{PWM} = \frac{F_{CPU}}{N \times 256}$$

where

$$\begin{aligned} F_{CPU} &= 1\text{MHz} \\ N &= 8 \end{aligned}$$

Therefore,

$$\begin{aligned} f_{PWM} &= \frac{1000000}{8 \times 256} \\ f_{PWM} &= 488.28\text{Hz} \end{aligned}$$

Ambient LED brightness is controlled at 488 Hz PWM frequency, which is sufficiently high to eliminate visible flickering.

5.2.3 LDR Brightness Resolution:

ADC resolution:

$$Resolution = \frac{V_{ref}}{2^{10}}$$

$$V_{ref} = 5V$$

$$Resolution = \frac{5}{1024}$$

$$Resolution = 4.88mV$$

Thus every ADC count represents $4.88mV$

ADC Range:

$$0 - 1023$$

PWM Range:

$$0 - 255$$

Therefore:

$$1024/256 = 4$$

Every 4 ADC counts correspond to 1 PWM level.

5.2.4 Ambient Brightness Mapping:

For bright environment,

$$ADC = 1023$$

$$Brightness = 255 - \frac{1023}{4}$$

$$Brightness = 0$$

PWM Duty 0%

For dark environment,

$$ADC = 0$$

$$Brightness = 255$$

PWM Duty 100%

For medium light,

$$\text{ADC} = 512$$

$$\text{Brightness} = 255 - \frac{512}{4}$$

$$\text{Brightness} = 127$$

Duty Cycle:

$$\frac{127}{255} \times 100$$

$$49.8\%$$

5.2.5 Fall Detection Timing Analysis:

Detection Time:

$$1000 \times 10\text{ms}$$

$$= 10000\text{ms}$$

$$= 10\text{s}$$

Therefore the system classifies inactivity after:

$$10\text{s}$$

5.2.6 Emergency Alert Timing:

Additional buzzer period:

$$500 \times 10\text{ms}$$

$$= 5000\text{ms}$$

$$= 5\text{s}$$

Total fall confirmation time $10 + 5 = 15\text{s}$

5.2.7 LDR Sampling Rate:

Sampling Interval:

$$10 \times 10\text{ms}$$

$$100\text{ms}$$

Sampling Frequency:

$$f = \frac{1}{0.1}$$

$$f = 10\text{Hz}$$

Thus the ambient light level is updated 10 times per second.

5.2.8 Staircase Hold Time:

After user exits:

$$3000ms = 3s$$

Both LEDs remain ON for 3s before shutdown.

5.2.9 Servo Motor Operation Analysis:

Pulse width range:

$$\begin{aligned} &2000 - 1000 \\ &= 1000\mu s \end{aligned}$$

Angular movement:

$$180^\circ$$

Angular resolution:

$$\begin{aligned} &\frac{180}{1000} \\ &0.18^\circ/\mu s \end{aligned}$$

5.2.10 Password Security Analysis:

Password length:

$$3$$

Possible symbols:

$$\{1, 2, 3\}$$

Total combinations:

$$\begin{aligned} &3^3 \\ &27 \end{aligned}$$

Thus 27 unique passwords can be generated.

6. Discussion:

The developed smart staircase safety and access control system successfully integrates multiple functionalities, including password-based authentication, adaptive illumination, fall detection, and remote emergency notification, into a unified embedded platform[10]. Experimental implementation and testing demonstrated that all subsystems operated reliably and interacted effectively under various operating conditions.

The password authentication subsystem achieved secure access control using only four push buttons and an OLED display. The use of three dedicated buttons for password entry simplified hardware implementation while maintaining sufficient security for a residential prototype. The OLED display provided immediate visual feedback to the user, reducing ambiguity during password entry and password modification procedures[2]. The servo motor effectively simulated a practical door-locking mechanism by providing smooth rotational movement between locked and unlocked positions. The complete door operation cycle, including opening, holding, and closing, required approximately seven seconds, which was found to be adequate for normal residential use.

The staircase monitoring subsystem demonstrated reliable movement detection capabilities. The combination of PIR sensors and IR break-beam sensors significantly enhanced detection accuracy compared to using a single sensing technology. PIR sensors alone are often susceptible to environmental disturbances and may occasionally produce false triggers due to temperature variations or moving heat sources. The addition of IR break-beam sensors allowed the system to validate actual entry and exit events, thereby minimizing erroneous detections and improving overall system robustness[3][4].

One of the most important features of the proposed design is the predictive lighting mechanism. Instead of illuminating the entire staircase continuously, the system selectively activates lighting zones in the user's direction of movement[5]. This approach not only enhances user comfort but also contributes substantially to energy conservation. The ambient brightness adjustment algorithm, implemented through an LDR sensor and PWM control, enables the system to respond dynamically to environmental lighting conditions[8]. Experimental calculations showed that the PWM signal operates at approximately 488 Hz, which is sufficiently high to prevent visible flickering while ensuring smooth brightness transitions. The LDR-based control updates illumination levels ten times per second, allowing rapid adaptation to changing ambient conditions.

The fall detection subsystem constitutes the most significant safety enhancement of the project. Continuous monitoring of user movement using mid-stair PIR sensors enables the system to identify prolonged inactivity, which may indicate a fall or medical emergency[3]. The implemented two-stage alert mechanism was designed to reduce false emergency notifications. In the first stage, a local buzzer warning is activated after ten seconds of inactivity, allowing the user an opportunity to resume movement or cancel the alert condition. If no motion is detected during

the subsequent five-second warning period, an emergency notification is transmitted to the caregiver through the ESP32-S3 and Blynk IoT platform. Based on timing analysis, the total fall confirmation duration was approximately fifteen seconds, which represents a reasonable trade-off between rapid emergency response and avoidance of false alarms[10].

Communication between the ATmega32A and ESP32-S3 modules was established through UART serial transmission at 4800 bps. Calculations revealed a baud rate error of only approximately 0.16%, ensuring highly reliable data exchange between the two microcontrollers[9]. Furthermore, the transmission time required to send emergency messages was estimated to be less than 30 milliseconds, indicating that the serial communication process contributes negligibly to the overall emergency response delay. The ESP32-S3 module maintained stable connectivity with the Blynk platform and successfully delivered notifications to the smartphone interface during testing[6][7].

Despite its satisfactory performance, several limitations were identified during system evaluation. The current password scheme permits only twenty-seven possible combinations because of the use of three push-button digits. Although this configuration is adequate for demonstrating proof-of-concept functionality, stronger authentication methods such as matrix keypads, RFID technology, fingerprint recognition, or facial recognition could significantly improve security in practical deployments. Additionally, PIR-based fall detection primarily relies on motion absence and may not distinguish between intentional inactivity and an actual fall event. Future improvements may incorporate accelerometers, pressure sensors, computer vision techniques, or machine learning algorithms to increase detection accuracy and reduce false positives[10].

Overall, the proposed system demonstrates the feasibility of combining embedded security mechanisms, intelligent energy management, and IoT-based healthcare monitoring within a cost-effective microcontroller architecture. The successful integration of ATmega32A, ESP32-S3, OLED display, PIR sensors, IR sensors, LDR, servo motor, and Blynk platform establishes a strong foundation for the development of next-generation smart home assistance systems aimed at improving residential safety, accessibility, and energy efficiency.

7. Expenditure:

SL no	Name of the components	Price	Quantity	Total
1	Lithium ion 3s	180	3	540
2	ATmega 32	430	1	430
3	AVR Programmer	1600	1	1600
4	HC-SR501 PIR Sensors	90	2	180
5	IR Sensor	50	2	100
6	LDR + Resistor	40	1	40
7	LED	10	4	40
8	OLED	250	1	250
9	ESP32 Module	450	1	450
10	Buzzer	50	1	50
11	LM2596 Buck Converter	100	1	100
12	Servo motor	150	1	150
13	Push Buttons	10	4	40
14	Bread Board	100	2	200
Total				4170

8. Application:

This integrated system has wide applications in modern smart infrastructure, particularly in environments where automation, safety, and security are required simultaneously.

A. Smart Home Security System

The ATmega32A-based password-controlled door lock provides a secure entry mechanism using a digital PIN system. This eliminates the need for traditional keys, reducing the risk of unauthorized access. The OLED display improves user interaction by providing real-time feedback during password entry and reset operations. Such systems are highly applicable in smart homes, apartments, and secure office spaces.

B. Assisted Living and Elderly Care Facilities

The staircase monitoring and safety system is highly beneficial for elderly individuals or patients with mobility challenges. PIR sensors and IR break-beam sensors detect movement direction and presence, ensuring safe navigation through stairs. In case of inactivity or potential falls, the system triggers a buzzer alert followed by a remote notification via ESP8266, allowing caregivers to respond quickly.

C. Intelligent Staircase Lighting System

The system dynamically controls LED zones based on user movement direction, ensuring that only the required sections of the staircase are illuminated. Combined with LDR-based ambient light sensing, the system automatically adjusts brightness levels according to environmental conditions. This makes it suitable for energy-efficient smart buildings, reducing unnecessary power consumption.

D. Safety and Fall Detection System

The inclusion of mid-stair PIR sensors enables continuous motion tracking. If movement suddenly stops for a predefined duration, the system assumes a possible fall or emergency situation. It then activates a two-stage alert system (local buzzer followed by IoT notification). This feature is especially useful in hospitals, rehabilitation centers, and elder care homes.

E. IoT-Based Remote Monitoring

With ESP8266 Wi-Fi integration and Blynk platform connectivity, the system enables real-time remote monitoring. Caregivers or homeowners can receive instant alerts on smartphones, making the system suitable for modern IoT-based smart surveillance and safety applications.

F. Energy-Efficient Smart Infrastructure

By combining motion detection, adaptive lighting, and ambient light sensing, the system ensures minimal energy wastage. Lights operate only when needed and adjust brightness automatically, making it suitable for green buildings and sustainable smart city projects.

9. Conclusion:

This project successfully designed and implemented an intelligent staircase safety and smart access control system using the ATmega32A microcontroller. It integrates password-based door locking, smart staircase lighting, ambient light adaptation, fall detection, and IoT-based emergency notification into a single embedded platform.

The password subsystem ensures secure access with push-button input, OLED feedback, and servo-controlled locking. The staircase monitoring system uses PIR and IR sensors to detect movement direction and control LED zones for improved visibility. An LDR-based PWM lighting control further optimizes energy efficiency under varying ambient light conditions.

A key feature is the two-stage fall detection system, which monitors inactivity, triggers a local buzzer alarm, and sends emergency alerts via the ESP32-S3 and Blynk IoT platform, making it suitable for elderly and mobility-impaired users.

Overall, the system demonstrates that low-cost microcontrollers and sensors can be effectively combined to build a reliable, energy-efficient, and scalable smart home safety solution. Future

improvements may include biometric authentication, advanced fall detection algorithms, and cloud-based data logging.

10. References:

- [1] Microchip Technology Inc., *ATmega32A Datasheet*, 8-bit AVR Microcontroller, 2021.
- [2] Solomon Systech Ltd., *SSD1306 OLED Display Controller Datasheet*, 2012.
- [3] Revomec Electronics, *HC-SR501 PIR Motion Sensor Datasheet*, 2018.
- [4] Generic IR Sensor Module Documentation, *Infrared Break Beam Sensor Technical Guide*, 2019.
- [5] J. B. Gupta, *Electronic Devices and Circuits*, S. K. Kataria & Sons, 2017. (LDR principles)
- [6] Espressif Systems, *ESP8266EX Datasheet*, 2020.
- [7] Blynk Inc., *Blynk IoT Platform Documentation*, 2023.
- [8] M. A. Mazidi, S. N. Mazidi, and B. McKinlay, *The AVR Microcontroller and Embedded Systems Using C*, Pearson, 2011. (PWM theory)
- [9] J. Axelson, *Serial Port Complete: COM Ports, USB Virtual COM Ports, and Ports for Embedded Systems*, Lakeview Research, 2007. (UART communication)
- [10] A. Al-Fuqaha et al., “Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications,” *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.
- [11] K. E. Årzén and D. Henriksson, *Embedded Systems Design: A Unified Hardware/Software Introduction*, Springer, 2019.

11. Appendix:

11.1 Code:

1. Staircase code using aATMega32:

```
/*
 * staircase_ir_2pir_buzzer_uart_ldr_atmega32a.c
 *
 * ATmega32A – Smart Staircase + LDR Auto-Brightness Ambient LED
 * =====
 *
 * Hardware connections
 * _____
 *
 * IR Sensor 1 (Entry)  OUT -> PA0 (input, pull-up, active LOW)
 * IR Sensor 2 (Exit)   OUT -> PA1 (input, pull-up, active LOW)
```

- * LDR (voltage divider) OUT -> PA2 (ADC2 input)
- *
 - * VCC
 - * |
 - * LDR
 - * |
 - * +-----> PA2 (ADC2)
 - * |
 - * 10k (fixed resistor)
 - * |
 - * GND
- *
 - * Stair LED1 -> PD0 (output, entry side)
 - * Stair LED2 -> PC0 (output, exit side — moved from PD1)
 - * Ambient LED (PWM) -> PD5 (OC1A, Timer1 Fast PWM 8-bit)
 - * 220Ω resistor to GND
 - * ON at power-up, brightness auto-set by LDR
- *
 - * PIR Sensor 1 OUT -> PD2 (INT0 input, no pull-up)
 - * PIR Sensor 2 OUT -> PD3 (INT1 input, no pull-up)
 - * PIR1 motion LED -> PB1 (output, 220Ω to GND)
 - * PIR2 motion LED -> PB2 (output, 220Ω to GND)
 - * Buzzer -> PB0 (output, active HIGH)
 - * UART TX -> PD1 (to ESP32 GPIO16/RX2 via voltage divider)
- *
 - * LDR Brightness Logic
 - * _____
 - * Bright light -> LDR resistance LOW -> ADC reading HIGH -> LED brightness LOW
 - * Dark -> LDR resistance HIGH -> ADC reading LOW -> LED brightness HIGH
 - * (Matches the voltage divider wiring above — LDR on top, fixed resistor to GND)
- *
 - * Staircase Behaviour
 - * _____
 - * IR1 fires → LED1 ON, PIRs enabled, wait for IR2 exit
 - * IR2 fires → LED2 ON, PIRs enabled, wait for IR1 exit
 - * Fall detected (10s no motion) → buzzer ON
 - * Buzzer beeps 5s with no response → send "FALL_DETECTED\n" via UART
 - * Normal exit via opposite IR → send "EXIT_OK\n" via UART
- *
 - * Timers used

```

* _____
* Timer1 (16-bit) → Fast PWM 8-bit on OC1A (PD5) for ambient LED brightness
* Timer0 is FREE
*
* Compile:
* avr-gcc -mmcu=atmega32 -DF_CPU=1000000UL -Os \
*     -o staircase.elf staircase_ir_2pir_buzzer_uart_ldr_atmega32a.c
* avr-objcopy -O ihex staircase.elf staircase.hex
* avrdude -c usbasp -p m32 -U flash:w:staircase.hex
*/

#define F_CPU 1000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <string.h>

/* _____ Pin definitions
_____ */
#define IR1_BIT    PA0    /* Entry IR sensor (active LOW)    */
#define IR2_BIT    PA1    /* Exit IR sensor (active LOW)    */
/* PA2 = ADC2 = LDR input (no #define needed, used as channel number 2)    */

#define LED1_BIT    PD0    /* Stair LED 1 – entry side    */
#define LED2_BIT    PC0    /* Stair LED 2 – exit side (moved from PD1) */
#define AMBIENT_BIT PD5    /* OC1A – ambient PWM LED (always on)    */

#define PIR1_BIT    PD2    /* PIR Sensor 1 OUT – INT0    */
#define PIR2_BIT    PD3    /* PIR Sensor 2 OUT – INT1    */
#define PIR1_LED_BIT PB1    /* PIR1 motion indicator LED    */
#define PIR2_LED_BIT PB2    /* PIR2 motion indicator LED    */
#define BUZZER_BIT  PB0    /* Buzzer (active HIGH)    */

/* _____ Tuning
_____ */
#define HOLD_MS      3000 /* All LEDs on hold after exit (ms)    */
#define NO_MOTION_LIMIT 1000 /* 10 sec = 1000 × 10 ms poll ticks    */
#define FALL_ALERT_LIMIT 500 /* 5 sec = 500 × 10 ms poll ticks    */

```

```

#define POLL_MS      10 /* Poll interval inside wait loop (ms) */

#define BAUD          4800
#define UBRR_VAL      ((F_CPU / (16UL * BAUD)) - 1)

/* — LDR update rate inside staircase wait loop — */
/* Every LDR_UPDATE_TICKS × POLL_MS ms the brightness is refreshed.
   At POLL_MS=10ms, 10 ticks = 100ms update rate — matches standalone code. */
#define LDR_UPDATE_TICKS  10

/* — Motion flags set by ISRs —
*/
volatile uint8_t motion1_flag = 0;
volatile uint8_t motion2_flag = 0;

/* —
ISR(s)

— */
ISR(INT0_vect) { motion1_flag = 1; }
ISR(INT1_vect) { motion2_flag = 1; }

/*
=====
=====
* ADC – LDR on PA2 (ADC2)
*
=====
===== */
static void adc_init(void)
{
    /* AVCC as reference, channel selected at read time */
    ADMUX = (1 << REFS0);
    /* Enable ADC, prescaler = 8 (1MHz / 8 = 125kHz — within 50–200kHz) */
    ADCSRA = (1 << ADEN) | (1 << ADPS1) | (1 << ADPS0);
}

static uint16_t adc_read(uint8_t channel)
{
    channel &= 0x07;
    ADMUX = (ADMUX & 0xF8) | channel; /* Select channel, keep ref bits */

```



```

    ADCSRA |= (1 << ADSC);          /* Start conversion          */
    while (ADCSRA & (1 << ADSC));    /* Wait for completion      */
    return ADC;                      /* 10-bit result (0–1023)   */
}

/*

=====

=====

* Timer1 – Fast PWM 8-bit on OC1A (PD5) for ambient LED
*

=====

===== */
static void pwm_init(void)
{
    DDRD |= (1 << AMBIENT_BIT);      /* PD5 as output          */
    /* Fast PWM 8-bit (WGM10 + WGM12), non-inverting OC1A (COM1A1),
       prescaler = 8 → PWM freq = 1MHz / (8 × 256) ≈ 488 Hz          */
    TCCR1A = (1 << COM1A1) | (1 << WGM10);
    TCCR1B = (1 << WGM12) | (1 << CS11);
}

static void pwm_set_duty(uint8_t duty)
{
    OCR1A = duty; /* 0 = LED off, 255 = full brightness          */
}

/*

=====

=====

* LDR brightness update — call this at ~100ms intervals
*

=====

===== */
static void ldr_update_brightness(void)
{
    uint16_t ldr_val = adc_read(2); /* Read ADC2 (PA2)          */
    /* Bright -> ADC high -> invert -> low brightness
       Dark   -> ADC low  -> invert -> high brightness          */
    uint8_t brightness = 255 - (uint8_t)(ldr_val >> 2);
    pwm_set_duty(brightness);
}

```

```

}

/*
=====
=====

* UART
*

=====
===== */
static void uart_init(void)
{
    UBRRH = (uint8_t)(UBRR_VAL >> 8);
    UBRRL = (uint8_t)(UBRR_VAL);
    UCSRB = (1 << TXEN);
    UCSRC = (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0); /* 8N1 */
}

static void uart_putc(char c)
{
    while (!(UCSRA & (1 << UDRE)));
    UDR = c;
}

static void uart_puts(const char *s)
{
    while (*s) uart_putc(*s++);
}

/*
=====
=====

* PIR helpers
*

=====
===== */
static inline void pir_enable(void)
{
    motion1_flag = 0;
    motion2_flag = 0;
    GIFR |= (1 << INTF0) | (1 << INTF1);
}

```

```

    GICR |= (1 << INT0) | (1 << INT1);
}

static inline void pir_disable(void)
{
    GICR &= ~((1 << INT0) | (1 << INT1));
    motion1_flag = 0;
    motion2_flag = 0;
}

/*
=====
=====
* Turn OFF staircase outputs (does NOT touch ambient PWM LED)
*
=====
===== */

static inline void all_off(void)
{
    PORTD &= ~(1 << LED1_BIT);
    PORTC &= ~(1 << LED2_BIT);
    PORTB &= ~((1 << PIR1_LED_BIT) | (1 << PIR2_LED_BIT) | (1 << BUZZER_BIT));
}

/*
=====
=====
* Wait for exit IR — with PIR monitoring, fall detection,
* UART alert, AND continuous LDR brightness updates
*
=====
===== */

static void wait_for_exit_ir(uint8_t exit_bit)
{
    uint16_t no_motion_ticks = 0;
    uint16_t buzzer_ticks = 0;
    uint8_t fall_confirmed = 0;
    uint8_t ldr_tick_count = 0; /* Counts up to LDR_UPDATE_TICKS */

    while (PINA & (1 << exit_bit))

```

```

{
    uint8_t motion_detected = 0;

    /* — PIR checks — */
    if (motion1_flag)
    {
        motion1_flag = 0;
        motion_detected = 1;
        PORTB |= (1 << PIR1_LED_BIT);
    }
    if (motion2_flag)
    {
        motion2_flag = 0;
        motion_detected = 1;
        PORTB |= (1 << PIR2_LED_BIT);
    }

    /* — LDR brightness refresh every 100ms — */
    ldr_tick_count++;
    if (ldr_tick_count >= LDR_UPDATE_TICKS)
    {
        ldr_tick_count = 0;
        ldr_update_brightness();
    }

    /* — Fall detection logic — */
    if (motion_detected)
    {
        no_motion_ticks = 0;
        buzzer_ticks = 0;
        fall_confirmed = 0;
        PORTB &= ~(1 << BUZZER_BIT);
    }
    else
    {
        no_motion_ticks++;

        if (no_motion_ticks >= NO_MOTION_LIMIT)
        {
            PORTB |= (1 << BUZZER_BIT);
        }
    }
}

```

```

        no_motion_ticks = NO_MOTION_LIMIT; /* Hold at limit */

        if (!fall_confirmed)
        {
            buzzer_ticks++;
            if (buzzer_ticks >= FALL_ALERT_LIMIT)
            {
                uart_puts("FALL_DETECTED\n");
                fall_confirmed = 1;
                buzzer_ticks = FALL_ALERT_LIMIT;
            }
        }
    }
}

    _delay_ms(POLL_MS);
}

uart_puts("EXIT_OK\n");
}

/*
=====
=====
* Peripheral initialisation
*
=====
===== */
static void init(void)
{
    /* — PA0, PA1 – IR inputs with pull-ups; PA2 – ADC (no pull-up) — */
    DDRA &= ~(1 << IR1_BIT) | (1 << IR2_BIT) | (1 << PA2);
    PORTA |= (1 << IR1_BIT) | (1 << IR2_BIT); /* PA2 pull-up stays OFF */

    /* — PD0 – LED1, PD5 – ambient PWM LED (init'd by pwm_init) — */
    DDRD |= (1 << LED1_BIT);
    PORTD &= ~(1 << LED1_BIT);

    /* — PC0 – LED2 output — */
    DDRC |= (1 << LED2_BIT);

```

```

PORTC &= ~(1 << LED2_BIT);

/* — PD2, PD3 – PIR inputs, no pull-up — */
DDRD &= ~((1 << PIR1_BIT) | (1 << PIR2_BIT));
PORTD &= ~((1 << PIR1_BIT) | (1 << PIR2_BIT));

/* — PB0 – Buzzer, PB1/PB2 – PIR LEDs — */
DDRB |= (1 << BUZZER_BIT) | (1 << PIR1_LED_BIT) | (1 << PIR2_LED_BIT);
PORTB &= ~((1 << BUZZER_BIT) | (1 << PIR1_LED_BIT) | (1 << PIR2_LED_BIT));

/* — INT0 & INT1 – rising edge — */
MCUCR |= (1 << ISC01) | (1 << ISC00) | (1 << ISC11) | (1 << ISC10);
GICR &= ~((1 << INT0) | (1 << INT1));

/* — Peripheral inits — */
adc_init();
pwm_init();
uart_init();

/* — Set initial brightness from LDR before main loop starts — */
ldr_update_brightness();

sei();
}

/*
=====
=====
* Main
*
=====
===== */
int main(void)
{
    uint8_t idle_ldr_tick = 0;

    init();

    while (1)
    {

```

```

/* — FORWARD: IR1 fires first ————— */
if (!(PINA & (1 << IR1_BIT)))
{
    PORTD |= (1 << LED1_BIT);
    pir_enable();

    wait_for_exit_ir(IR2_BIT);

    pir_disable();
    PORTB &= ~(1 << BUZZER_BIT);
    PORTC |= (1 << LED2_BIT);

    _delay_ms(HOLD_MS);
    all_off();

    while (!(PINA & (1 << IR1_BIT)) || !(PINA & (1 << IR2_BIT)));
}

/* — REVERSE: IR2 fires first ————— */
else if (!(PINA & (1 << IR2_BIT)))
{
    PORTC |= (1 << LED2_BIT);
    pir_enable();

    wait_for_exit_ir(IR1_BIT);

    pir_disable();
    PORTB &= ~(1 << BUZZER_BIT);
    PORTD |= (1 << LED1_BIT);

    _delay_ms(HOLD_MS);
    all_off();

    while (!(PINA & (1 << IR1_BIT)) || !(PINA & (1 << IR2_BIT)));
}

/* — IDLE: no IR triggered — keep updating LDR brightness — */
else
{
    _delay_ms(POLL_MS);
}

```

```

        idle_ldr_tick++;
        if (idle_ldr_tick >= LDR_UPDATE_TICKS)
        {
            idle_ldr_tick = 0;
            ldr_update_brightness();
        }
    }
}

return 0;
}

```

2. Password protected door code using ATmega32:

```

#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <string.h>

// ----- Font 5x7 -----
const uint8_t font5x7[][5] = {
    {0x00,0x00,0x00,0x00,0x00}, // space (0x20)
    {0x00,0x00,0x5F,0x00,0x00}, // !
    {0x00,0x07,0x00,0x07,0x00}, // "
    {0x14,0x7F,0x14,0x7F,0x14}, // #
    {0x24,0x2A,0x7F,0x2A,0x12}, // $
    {0x23,0x13,0x08,0x64,0x62}, // %
    {0x36,0x49,0x55,0x22,0x50}, // &
    {0x00,0x05,0x03,0x00,0x00}, // '
    {0x00,0x1C,0x22,0x41,0x00}, // (
    {0x00,0x41,0x22,0x1C,0x00}, // )
    {0x14,0x08,0x3E,0x08,0x14}, // *
    {0x08,0x08,0x3E,0x08,0x08}, // +
    {0x00,0x50,0x30,0x00,0x00}, // ,
    {0x08,0x08,0x08,0x08,0x08}, // -
    {0x00,0x60,0x60,0x00,0x00}, // .
    {0x20,0x10,0x08,0x04,0x02}, // /

```



```

{0x3E,0x51,0x49,0x45,0x3E}, // 0
{0x00,0x42,0x7F,0x40,0x00}, // 1
{0x42,0x61,0x51,0x49,0x46}, // 2
{0x21,0x41,0x45,0x4B,0x31}, // 3
{0x18,0x14,0x12,0x7F,0x10}, // 4
{0x27,0x45,0x45,0x45,0x39}, // 5
{0x3C,0x4A,0x49,0x49,0x30}, // 6
{0x01,0x71,0x09,0x05,0x03}, // 7
{0x36,0x49,0x49,0x49,0x36}, // 8
{0x06,0x49,0x49,0x29,0x1E}, // 9
{0x00,0x36,0x36,0x00,0x00}, // :
{0x00,0x56,0x36,0x00,0x00}, // ;
{0x08,0x14,0x22,0x41,0x00}, //
{0x14,0x14,0x14,0x14,0x14}, // =
{0x00,0x41,0x22,0x14,0x08}, // >
{0x02,0x01,0x51,0x09,0x06}, // ?
{0x32,0x49,0x79,0x41,0x3E}, // @
{0x7E,0x11,0x11,0x11,0x7E}, // A
{0x7F,0x49,0x49,0x49,0x36}, // B
{0x3E,0x41,0x41,0x41,0x22}, // C
{0x7F,0x41,0x41,0x22,0x1C}, // D
{0x7F,0x49,0x49,0x49,0x41}, // E
{0x7F,0x09,0x09,0x09,0x01}, // F
{0x3E,0x41,0x49,0x49,0x7A}, // G
{0x7F,0x08,0x08,0x08,0x7F}, // H
{0x00,0x41,0x7F,0x41,0x00}, // I
{0x20,0x40,0x41,0x3F,0x01}, // J
{0x7F,0x08,0x14,0x22,0x41}, // K
{0x7F,0x40,0x40,0x40,0x40}, // L
{0x7F,0x02,0x0C,0x02,0x7F}, // M
{0x7F,0x04,0x08,0x10,0x7F}, // N
{0x3E,0x41,0x41,0x41,0x3E}, // O
{0x7F,0x09,0x09,0x09,0x06}, // P
{0x3E,0x41,0x51,0x21,0x5E}, // Q
{0x7F,0x09,0x19,0x29,0x46}, // R
{0x46,0x49,0x49,0x49,0x31}, // S
{0x01,0x01,0x7F,0x01,0x01}, // T
{0x3F,0x40,0x40,0x40,0x3F}, // U
{0x1F,0x20,0x40,0x20,0x1F}, // V
{0x3F,0x40,0x38,0x40,0x3F}, // W

```

```

    {0x63,0x14,0x08,0x14,0x63}, // X
    {0x07,0x08,0x70,0x08,0x07}, // Y
    {0x61,0x51,0x49,0x45,0x43} // Z
};

// ----- I2C (TWI) -----
#define SSD1306_ADDR 0x3C

void i2c_init(void)
{
    TWSR = 0x00;
    TWBR = 0x02; // ~100kHz @ 1MHz (minimum, will be slower; fine for OLED)
    TWCR = (1 << TWEN);
}

void i2c_start(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
    while (!(TWCR & (1 << TWINT)));
}

void i2c_stop(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTO) | (1 << TWEN);
    while (TWCR & (1 << TWSTO));
}

void i2c_write(uint8_t data)
{
    TWDR = data;
    TWCR = (1 << TWINT) | (1 << TWEN);
    while (!(TWCR & (1 << TWINT)));
}

// ----- SSD1306 -----
void ssd1306_cmd(uint8_t cmd)
{
    i2c_start();
    i2c_write(SSD1306_ADDR << 1);
    i2c_write(0x00);
}

```

```

    i2c_write(cmd);
    i2c_stop();
}

void ssd1306_data(uint8_t data)
{
    i2c_start();
    i2c_write(SSD1306_ADDR << 1);
    i2c_write(0x40);
    i2c_write(data);
    i2c_stop();
}

void ssd1306_init(void)
{
    _delay_ms(100);
    ssd1306_cmd(0xAE);
    ssd1306_cmd(0xD5); ssd1306_cmd(0x80);
    ssd1306_cmd(0xA8); ssd1306_cmd(0x3F);
    ssd1306_cmd(0xD3); ssd1306_cmd(0x00);
    ssd1306_cmd(0x40);
    ssd1306_cmd(0x8D); ssd1306_cmd(0x14);
    ssd1306_cmd(0x20); ssd1306_cmd(0x00);
    ssd1306_cmd(0xA1);
    ssd1306_cmd(0xC8);
    ssd1306_cmd(0xDA); ssd1306_cmd(0x12);
    ssd1306_cmd(0x81); ssd1306_cmd(0x80);
    ssd1306_cmd(0xD9); ssd1306_cmd(0xF1);
    ssd1306_cmd(0xDB); ssd1306_cmd(0x40);
    ssd1306_cmd(0xA4);
    ssd1306_cmd(0xA6);
    ssd1306_cmd(0xAF);
}

void ssd1306_set_cursor(uint8_t col, uint8_t page)
{
    ssd1306_cmd(0x21); ssd1306_cmd(col); ssd1306_cmd(127);
    ssd1306_cmd(0x22); ssd1306_cmd(page); ssd1306_cmd(7);
}

```

```

void ssd1306_clear(void)
{
    ssd1306_set_cursor(0, 0);
    for (uint16_t i = 0; i < 1024; i++)
        ssd1306_data(0x00);
}

void ssd1306_putchar(char c)
{
    if (c < 0x20 || c > 0x5A) c = ' ';
    uint8_t idx = c - 0x20;
    for (uint8_t i = 0; i < 5; i++)
        ssd1306_data(font5x7[idx][i]);
    ssd1306_data(0x00);
}

void ssd1306_print(const char *str)
{
    while (*str)
        ssd1306_putchar(*str++);
}

void ssd1306_clear_line(uint8_t page)
{
    ssd1306_set_cursor(0, page);
    for (uint8_t i = 0; i < 21; i++)
        ssd1306_print(" ");
}

// ----- Servo (PWM on PD5 / OC1A) -----
#define SERVO_POS0 1000 // 0 degrees
#define SERVO_POS180 2000 // 180 degrees

void PWM_init(void)
{
    DDRD |= (1 << PD5); // OC1A as output
    TCCR1A = (1 << COM1A1) | (1 << WGM11);
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS10); // Prescaler = 1
    ICR1 = 20000; // 20ms period @ 1MHz, prescaler 1
    OCR1A = SERVO_POS0; // start at 0 degrees
}

```

```

}

// ----- Buttons -----
// BTN1, BTN2, BTN3 -> PB0, PB1, PB2 (number entry)
// BTN4 -> PB3 (password reset trigger)
#define BTN1 PB0
#define BTN2 PB1
#define BTN3 PB2
#define BTN4 PB3

uint8_t debounce_btn(uint8_t pin)
{
    if (!(PINB & (1 << pin)))
    {
        _delay_ms(20);
        if (!(PINB & (1 << pin)))
        {
            while (!(PINB & (1 << pin))); // wait release
            _delay_ms(20);
            return 1;
        }
    }
    return 0;
}

// ----- Password storage -----
char password[4] = "123"; // default password, stored in RAM

// ----- Helper: read one digit (1/2/3) blocking -----
char read_digit(void)
{
    while (1)
    {
        if (debounce_btn(BTN1)) return '1';
        if (debounce_btn(BTN2)) return '2';
        if (debounce_btn(BTN3)) return '3';
        if (debounce_btn(BTN4)) return 'R'; // allow cancel during entry
    }
}

```

```

// ----- Main -----
int main(void)
{
    // Buttons PB0-PB3, input with pull-ups
    DDRB &= ~((1 << BTN1) | (1 << BTN2) | (1 << BTN3) | (1 << BTN4));
    PORTB |= (1 << BTN1) | (1 << BTN2) | (1 << BTN3) | (1 << BTN4);

    i2c_init();
    ssd1306_init();
    ssd1306_clear();

    PWM_init(); // servo on PD5 (OC1A), starts at 0 degrees

    ssd1306_set_cursor(0, 0);
    ssd1306_print("Enter Password:");

    char seq[4] = "";
    uint8_t idx = 0;

    while (1)
    {
        uint8_t pressed = 0;
        char digit = 0;

        // ---- Check BTN4 (reset trigger) first ----
        if (debounce_btn(BTN4))
        {
            idx = 0;
            seq[0] = '\0';

            ssd1306_clear_line(0);
            ssd1306_clear_line(2);
            ssd1306_clear_line(4);
            ssd1306_set_cursor(0, 0);
            ssd1306_print("Enter OLD Pass:");

            char oldSeq[4] = "";
            for (uint8_t i = 0; i < 3; i++)
            {
                char d = read_digit();

```

```

if (d == 'R')
{
    ssd1306_clear_line(0);
    ssd1306_clear_line(2);
    ssd1306_clear_line(4);
    ssd1306_set_cursor(0, 0);
    ssd1306_print("Enter Password:");
    goto continue_main;
}
oldSeq[i] = d;
oldSeq[i+1] = '\0';

ssd1306_set_cursor(0, 2);
ssd1306_print("Old: ");
ssd1306_print(oldSeq);
ssd1306_print("  ");
}

if (strcmp(oldSeq, password) == 0)
{
    ssd1306_clear_line(2);
    ssd1306_clear_line(4);
    ssd1306_set_cursor(0, 0);
    ssd1306_print("Enter NEW Pass:");

    char newSeq[4] = "";
    for (uint8_t i = 0; i < 3; i++)
    {
        char d = read_digit();
        if (d == 'R')
        {
            ssd1306_clear_line(0);
            ssd1306_clear_line(2);
            ssd1306_clear_line(4);
            ssd1306_set_cursor(0, 0);
            ssd1306_print("Enter Password:");
            goto continue_main;
        }
        newSeq[i] = d;
        newSeq[i+1] = '\0';
    }
}

```

```

        ssd1306_set_cursor(0, 2);
        ssd1306_print("New: ");
        ssd1306_print(newSeq);
        ssd1306_print("  ");
    }

    strcpy(password, newSeq);

    ssd1306_set_cursor(0, 4);
    ssd1306_print("Password Changed");
    _delay_ms(1500);
}
else
{
    ssd1306_set_cursor(0, 4);
    ssd1306_print("Wrong Password ");
    _delay_ms(1500);
}

ssd1306_clear_line(0);
ssd1306_clear_line(2);
ssd1306_clear_line(4);
ssd1306_set_cursor(0, 0);
ssd1306_print("Enter Password:");

idx = 0;
seq[0] = '\0';
}

// ---- Normal password entry ----
if (debounce_btn(BTN1)) { digit = '1'; pressed = 1; }
else if (debounce_btn(BTN2)) { digit = '2'; pressed = 1; }
else if (debounce_btn(BTN3)) { digit = '3'; pressed = 1; }

if (pressed)
{
    seq[idx] = digit;
    idx++;
    seq[idx] = '\0';
}

```



```

ssd1306_set_cursor(0, 2);
ssd1306_print("Sequence: ");
ssd1306_print(seq);
ssd1306_print("  ");

if (idx == 3)
{
    if (strcmp(seq, password) == 0)
    {
        ssd1306_set_cursor(0, 4);
        ssd1306_print("Access Granted ");

        // Servo: 0 -> 180
        OCR1A = SERVO_POS180;
        _delay_ms(1500);

        // Hold at 180 for 4 sec
        _delay_ms(4000);

        // Servo: 180 -> 0
        OCR1A = SERVO_POS0;
        _delay_ms(1500);

        ssd1306_set_cursor(0, 4);
        ssd1306_print("          ");
        ssd1306_set_cursor(0, 0);
        ssd1306_print("Enter Password: ");
    }
    else
    {
        ssd1306_set_cursor(0, 4);
        ssd1306_print("Access Denied ");
        _delay_ms(1000);
        ssd1306_set_cursor(0, 4);
        ssd1306_print("          ");
    }

    idx = 0;
    seq[0] = '\0';
}

```

```

        ssd1306_set_cursor(0, 2);
        ssd1306_print("Sequence:    ");
    }
}

    continue_main;;
}
}

```

3. Esp32 code for Blynk IoT:

```

/*
 * esp32_staircase_monitor.ino
 *
 * ESP32-S3 – Staircase Monitor with UART + WiFi + Blynk IoT
 * =====
 *
 * Wiring:
 * ATmega32A PD1 (TX) --> Voltage Divider --> ESP32-S3 GPIO16 (RX2)
 * ATmega32A GND    --> ESP32-S3 GND
 *
 * Messages received from ATmega32A:
 * "FALL_DETECTED\n" → LED ON  on Blynk
 * "EXIT_OK\n"      → LED OFF on Blynk
 *
 * Baud rate ATmega↔ESP32 : 4800
 * Serial Monitor baud    : 115200
 */

#define BLYNK_TEMPLATE_ID   "TMPL6Ob03wymj"
#define BLYNK_TEMPLATE_NAME "Atmega32"
#define BLYNK_AUTH_TOKEN    "ocglyclJo71iw65MsJcrZBdeDZxxeF3G"

#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

/* ——— WiFi credentials —————
 */
#define WIFI_SSID    "Edge 50"

```

```
#define WIFI_PASSWORD "@1234567890"
```

```
/*          —          UART          pins
_____ */
```

```
#define RXD2    16
#define TXD2    17
#define BAUD_RATE 4800
```

```
/*          —          Virtual          pin
_____ */
```

```
#define VPIN_LED V1  /* LED widget – ON on fall, OFF on exit */
```

```
String receivedMessage = "";
```

```
/*
=====
===== */
```

```
/* Setup          */
/*
```

```
===== */
```

```
void setup()
```

```
{
  Serial.begin(115200);
  delay(2000);
```

```
  Serial.println("=====");
  Serial.println(" Staircase Monitor - ESP32-S3 ");
  Serial.println(" Connecting to Blynk... ");
  Serial.println("=====");
```

```
  /* UART2 – receive from ATmega32A */
  Serial2.begin(BAUD_RATE, SERIAL_8N1, RXD2, TXD2);
```

```
  /* Connect WiFi + Blynk */
  Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASSWORD);
```

```
  Serial.println("=====");
  Serial.println(" WiFi + Blynk Connected! ");
  Serial.print (" IP Address: "); Serial.println(WiFi.localIP());
```

```

Serial.println("=====");

/* LED OFF at start */
Blynk.virtualWrite(VPIN_LED, 0);
}

/*
===== */
/* Loop */
/*
===== */
===== */
void loop()
{
  Blynk.run();

  /* — Read UART from ATmega32A — */
  while (Serial2.available())
  {
    char c = (char)Serial2.read();

    if (c == '\n')
    {
      receivedMessage.trim();
      handleMessage(receivedMessage);
      receivedMessage = "";
    }
    else
    {
      receivedMessage += c;
    }
  }
}

/*
===== */
===== */
/* Handle message from ATmega32A */

```

```

/*
===== */
void handleMessage(String msg)
{
    if (msg.length() == 0) return;

    Serial.print("[WiFi] ");
    Serial.println(WiFi.status() == WL_CONNECTED ? "Online" : "Offline");
    Serial.print("[ATmega32A] --> ");
    Serial.println(msg);

    if (msg == "FALL_DETECTED")
    {
        Serial.println("*****");
        Serial.println(" !! FALL DETECTED ON STAIRCASE !! ");
        Serial.println(" Person has not moved for 15 seconds. ");
        Serial.println(" Buzzer has been beeping for 5 seconds. ");
        Serial.println(" Immediate attention may be required! ");
        Serial.println("*****");
        Serial.println();

        Blynk.virtualWrite(VPIN_LED, 255);    /* LED ON */
        Blynk.logEvent("fall_detected", "Person may have fallen on staircase!");
    }
    else if (msg == "EXIT_OK")
    {
        Serial.println("-----");
        Serial.println(" Person exited staircase safely. ");
        Serial.println(" System reset and ready. ");
        Serial.println("-----");
        Serial.println();

        Blynk.virtualWrite(VPIN_LED, 0);    /* LED OFF */
    }
    else
    {
        Serial.print(" [Unknown message received]: ");
        Serial.println(msg);
    }
}

```

}

11.2 Abbreviations:

- **OLED** → Organic Light Emitting Diode (Display)
- **PIR** → Passive Infrared (Sensor)
- **IR** → Infrared
- **LDR** → Light Dependent Resistor
- **ADC** → Analog-to-Digital Converter
- **PWM** → Pulse Width Modulation
- **Wi-Fi** → Wireless Fidelity (Wireless Networking Technology)
- **LED** → Light Emitting Diode
- **Blynk** → IoT (Internet of Things) Cloud Platform for remote monitoring/control
- **IoT** → Internet of Things
- **MCU** (implied from context) → Microcontroller Unit
- **GPIO** (general embedded implied concept, if referenced elsewhere) → General Purpose Input Output